

CS6135 VLSI Physical Design Automation

Homework 4: Global Routing

賈俊佑 studentID:114062645

1.Introduction

Given the routing grid size, edge capacities, and a set of 2-pin nets, implement a grid-based global router to generate routing paths for a given circuit. The routing process should consider edge congestion and capacity constraints while producing a valid routing solution for all nets.

2.Objective

The global routing problem is to determine a routing path for each net n_i on a grid-based routing graph. Each routing path connects the two pins of the net using horizontal and vertical edges, forming a Manhattan routing solution.

Each edge e has a fixed routing capacity $s(e)$, representing the number of available routing tracks. The demand $d(e)$ denotes the number of nets passing through edge e . The overflow on an edge is defined as:

$$overflow(e) = \begin{cases} d(e) - s(e), & \text{if } d(e) > s(e) \\ 0, & \text{otherwise} \end{cases}$$

The primary objective is to minimize the total overflow over all edges, and the secondary objective is to minimize the total wirelength of all routing paths.

$$\min \sum_{e \in E} overflow(e)$$

$$\min \sum_{n_i \in N} \sum_{e \in E_i} length(e)$$

3. Result

```
+-----+
|       This script is used for PDA HW4 grading.       |
+-----+
host name: ic21
compiler version: g++ (GCC) 9.3.0

grading on 114062645:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | overflow | wirelength | runtime | status
+-----+
public1 | 0 | 7061 | 0.02 | success
public2 | 0 | 304606 | 0.48 | success
public3 | 0 | 1029440 | 1.47 | success
public4 | 0 | 2268642 | 16.67 | success
+-----+

Successfully write grades to HW4_grade.csv
+-----+
```

4. Method

4.1 Overall Flow

The overall routing flow is illustrated as follows:

1. Initial Stage
Generate a fast and reasonable initial solution using monotonic routing under bounding-box constraints.
2. Main Stage
Iteratively resolve routing overflows using A*-based maze routing with congestion-aware and history-based cost functions.
3. Refinement Stage
After all overflows are eliminated, further reduce total wirelength while strictly preserving routability.

The overall procedure can be summarized by the following pseudocode:

```
GlobalRoute():  
    RunInitialStage()  
    RunMainStage()  
    RunRefinementStage()
```

This staged design allows the router to first prioritize routability and congestion resolution and then focus on wirelength optimization once a feasible solution is obtained.

4.2 Initial Stage

4.2.1 Routing Method

In the initial stage, we apply monotonic routing to quickly generate an initial routing solution.

For each two-pin net, routing is restricted to move only in the directions that monotonically approach the target pin. Specifically:

- If the source and target pins differ in the x-direction, movement in x is restricted to only increase or decrease toward the target.
- Similarly, movement in y is restricted toward the target.
- The search space is limited to the net's bounding box.

This effectively performs a shortest path search similar to Dijkstra's algorithm within a constrained region, significantly reducing runtime while avoiding unnecessary detours.

The search can be expressed in the standard A* form:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the accumulated routing cost and $h(n)$ is the heuristic. In the initial stage, we use $h(n) = 0$, thus the search degenerates to Dijkstra's algorithm under the monotonic and bounding-box constraints. The accumulated cost is computed as:

$$g(n) = \sum_{e \in P(s \rightarrow n)} C_{initial}(e)$$

where $P(s \rightarrow n)$ denotes the current partial path from source s to node n , and $C_{initial}(e)$ is defined in Section 4.2.3.

4.2.2 Net Ordering Strategy

Nets are divided into two categories:

- Flat nets: nets whose pins share the same x or y coordinate.
- Standard nets: all remaining nets.

Flat nets are routed first.

Standard nets are sorted by:

1. Bounding-box area (ascending)
2. Half-perimeter wirelength (HPWL) (ascending)

This ordering prioritizes nets with smaller routing freedom, reducing early congestion in tight regions.

4.2.3 Cost Function in Initial Stage

The cost of traversing an edge is computed based on its utilization:

Let:

- d be the edge demand after routing,
- c be the edge capacity,
- $u = \frac{d}{c}$ be the utilization ratio.

The edge cost in the initial stage consists of three components:

(1) Base Cost

A base cost is applied to favor short paths while mildly penalizing higher utilization:

$$C_{base}(e) = 1 + \alpha \cdot u^{k_b}$$

where:

- $\alpha = 0.6$ is the base utilization weighting factor,
- $k_b = 3$ controls the nonlinearity of utilization growth.

This term introduces a smooth, super-linear penalty on utilization, encouraging the router to avoid highly utilized edges even before congestion becomes critical.

(2) Utilization Penalty ($u > T$)

To further discourage routing through edges that are close to saturation, an additional utilization penalty is applied when utilization exceeds a predefined threshold T :

$$C_{util}(e) = \begin{cases} \beta \cdot \left(\frac{u - T}{1 - T}\right)^{k_u}, & u > T, \\ 0, & \text{otherwise.} \end{cases}$$

where:

- $T = 0.82$ is the utilization threshold,
- $\beta = 12$ controls the strength of the penalty,
- $k_u = 4$ determines how sharply the penalty increases beyond the threshold.

This term is activated only when an edge is close to full capacity, allowing normal routing at low utilization while aggressively penalizing near-saturated edges.

(3) Overflow Penalty ($d > c$)

To strongly discourage overflow while still allowing limited violations in the initial stage, a quadratic overflow penalty is applied when the demand exceeds capacity:

$$C_{overflow}(e) = \begin{cases} \gamma \cdot (d - c)^2, & d > c, \\ 0, & \text{otherwise.} \end{cases}$$

where:

- $\gamma = 35$ is the overflow penalty coefficient.

The quadratic form ensures that small overflows are tolerated, but large overflows are penalized very aggressively.

Final Edge Cost (Initial Stage)

$$C_{initial}(e) = C_{base}(e) + C_{util}(e) + C_{overflow}(e)$$

This formulation allows limited congestion during the initial stage while strongly penalizing severe overflow, enabling the router to quickly obtain a feasible starting solution.

4.3 Main Stage

4.3.1 Routing Method

The main stage employs A*-based maze routing to iteratively reduce routing overflow.

- Search graph: 2D grid points
- Neighbor expansion: four directions (up, down, left, right)
- Priority function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the accumulated routing cost,
- $h(n)$ is the Manhattan distance to the target.

In our implementation, $g(n)$ is the accumulated path cost, computed as the sum of edge costs along the partial path from the source to node n . In the main stage, each traversed edge e contributes C_{main} is defined in Section 4.3.4. Therefore,

$$g(n) = \sum_{e \in P(s \rightarrow n)} C_{main}(e)$$

where $P(s \rightarrow n)$ denotes the current partial path from the source s to node n .

This approach allows efficient exploration of congestion-aware shortest paths.

4.3.2 Candidate Selection

In each iteration, candidate nets include:

- Nets that are currently unrouted
- Nets whose existing routes pass through overflow edges

Candidates are sorted by:

1. Number of overflow edges on the path (descending)
2. Bounding-box area (descending)
3. HPWL (descending)

This prioritizes nets that contribute most to congestion.

4.3.3 History-Based Congestion Update

To prevent oscillation, a history cost is maintained for each edge.

- After each iteration, edges with overflow increment their history value.
- History cost increases the penalty of repeatedly congested edges, encouraging alternative routing paths.

4.3.4 Cost Function in Main Stage

(1) Under-Capacity Cost ($d \leq c$)

For an edge whose new demand does not exceed its capacity, apply a linear utilization-based cost to encourage early load balancing:

$$C_{under}(e) = 1 + \delta \cdot r$$

where:

- $r = \frac{d}{c}$ is the utilization ratio,
- $\delta = 0.5$ is a constant weighting factor.

This term provides a smooth load-balancing effect before overflow occurs.

(2) Over-Capacity Cost ($d > c$)

For a congested edge (i.e., $d > c$), the cost incorporates utilization, iteration count, and a history penalty:

$$C_{over}(e) = 1 + (k_{slope} + k_{iter} \cdot i) \cdot H(e) \cdot r^{k(i)}$$

where:

- $k_{slope} = 1.0$
- $k_{iter} = 0.05$
- i is the current iteration index
- $k(i) = \min(5.0, 2.5 + k_{iter} \cdot i)$
- $H(e)$ is the overflow history value of edge e .

These parameters progressively increase congestion penalties over iterations, enabling the router to transition from feasibility-oriented routing to aggressive congestion resolution.

Final Edge Cost (Main Stage)

$$C_{main}(e) = \begin{cases} C_{under}, & d \leq c \\ C_{over}, & d > c \end{cases}$$

This adaptive cost formulation gradually increases congestion penalties over iterations, guiding the solution toward a zero-overflow routing.

4.4 Refinement Stage

4.4.1 Objective

Once all routing overflows are eliminated, the refinement stage aims to reduce the total wirelength while strictly preserving routability.

Only nets whose current routing path length exceeds their half-perimeter wirelength (HPWL) are considered, since such nets have potential for further wirelength reduction.

4.4.2 Routing Strategy

The refinement stage adopts the same A*-based maze routing framework as the main stage, but with a strict overflow-blocking cost function.

Routing is performed on the same 2D grid graph with four-directional neighbor expansion (up, down, left, right).

The A* priority function is defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the accumulated routing cost from the source to node n ,
- $h(n)$ is the Manhattan distance to the target pin:

In the refinement stage, $g(n)$ is computed as the sum of refinement-stage edge costs along the partial path:

$$g(n) = \sum_{e \in P(s \rightarrow n)} C_{ref}(e)$$

where $P(s \rightarrow n)$ denotes the partial routing path from the source s to node n .

4.4.3 Candidate Selection

In the refinement stage, candidate nets include:

- Nets whose current routing path length strictly exceeds their Half-Perimeter Wirelength (HPWL).

Candidates are sorted by:

1. Difference between current path length and HPWL (descending)

2. Bounding-box area (descending)
3. HPWL (descending)

This prioritizes nets with the highest potential for wirelength reduction, targeting those with significant detours first.

4.4.4 Cost Function in Refinement Stage

For each edge e , the routing cost is defined as:

$$C_{ref}(e) = \begin{cases} 10^9, & d > c, \\ 1, & d \leq c. \end{cases}$$

where:

- d is the new edge demand after routing,
- c is the edge capacity.

This cost formulation effectively prohibits the A* search from selecting any edge that would cause overflow, thereby constraining the search space to overflow-free routing solutions.

4.4.5 Acceptance Criterion

A new route is accepted only if:

- It does not introduce overflow, and
- It strictly reduces wirelength compared to the original route.

5. Question

Did you implement rip-up and re-route?

✓ If yes, explain how you determined which nets to rip up and the order in which they were re-routed.

✓ If not, describe what methods you used to reduce routing overflow instead.

Yes. I implemented a rip-up and re-route procedure in the main routing stage. The router iteratively identifies nets that contribute to overflow,

removes their existing routes, and re-routes them using congestion-aware maze routing.

In each iteration, I construct a candidate list consisting of:

1. All unrouted nets, which must be routed first.
2. Routed nets that traverse at least one overflow edge. For each routed net, I scan the edges along its current path and mark the net as a rip-up candidate if any edge on the path is currently overflow (i.e., demand exceeds capacity). This targets nets that directly contribute to congestion instead of re-routing everything.

Re-route order:

The candidate nets are sorted in a priority order before re-routing:

1. Number of overflow edges on the current path (descending) — nets causing more overflow are handled earlier.
2. Bounding box area (descending) — larger nets are prioritized since they typically have broader impact on congestion.
3. HPWL (descending) — longer nets are routed earlier as they are harder to accommodate later.

Rip-up and re-route procedure:

The candidate nets are processed sequentially, one at a time. For each candidate, the operation is atomic:

1. I first rip up its current path by decrementing edge demand along that path.
2. Then, I re-route the net using maze routing ($\$A^*\$$ -based) with a congestion-aware edge cost.
3. I immediately commit the result by incrementing edge demand along the new path before processing the next candidate.

This ensures that every net is routed based on the most up-to-date congestion information. The cost function reflects current demand/capacity and also includes a history term so that edges that stay congested across iterations become progressively more expensive. This iterative process continues until overflow stops improving or the time/iteration limit is reached.

6. Conclusion: Learnings and Challenges

Through this project, I gained a deeper understanding of the global routing problem and the practical challenges involved in balancing routability, congestion, and wirelength. One of the key learnings was that

routing quality is highly dependent on the design of the cost function rather than the routing algorithm alone. Even with the same maze routing framework, different cost formulations can lead to significantly different congestion behaviors and routing outcomes.

In this implementation, the cost function was designed to reflect both current edge demand and historical congestion, taking inspiration from prior global routing approaches described in the literature [1][2]. By incorporating demand-to-capacity ratios and history-based penalties, the router is encouraged to avoid repeatedly congested edges and gradually escape local congestion hotspots. This experience highlighted how reference designs can guide cost modeling, while still requiring careful adaptation to fit a specific routing framework.

Another important challenge was determining when and how to apply rip-up and re-route effectively. Aggressive rip-up strategies can increase runtime and instability, while overly conservative strategies may fail to reduce overflow. Through experimentation, I learned the importance of selectively targeting nets that directly contribute to overflow and ordering them appropriately during re-routing.

Overall, this project strengthened my understanding of congestion-aware routing, iterative optimization, and cost function design. It demonstrated that effective global routing is not only about finding paths, but also about encoding routing priorities and trade-offs into the cost model in a principled and adaptive manner.

7. Declaration on the Use of AI Tools

In this project, I used AI tools including Google's Gemini and OpenAI's ChatGPT, for two primary purposes:

1. Debugging and Code Verification:

I consulted the AI to help inspect portions of my C++ implementation related to maze routing, rip-up and re-route logic, and edge demand updates. This assistance was particularly useful for identifying potential logical errors.

2. Algorithm Design and Code Refinement:

I used the AI to discuss high-level algorithmic strategies for global

routing, including congestion-aware cost modeling and the selection of candidate nets for rip-up and re-route. The AI was also used to refine code structure and improve clarity without altering the core algorithmic decisions or bypassing the problem-solving process.

All design decisions, algorithm implementation, and final code were independently developed and validated by me. The AI tools served solely as a supplementary resource for debugging, reasoning, and improving code readability.

8. References

- [1] Y.-J. Chang, Y.-T. Lee, and T.-C. Wang, "NTHU-Route 2.0: A Fast and Stable Global Router," in Proceedings of International Conference on Computer-Aided Design, 2008.
- [2] Y.-J. Chang, Y.-T. Lee, J.-R. Gao, P.-C. Wu, and T.-C. Wang, "NTHU-Route 2.0: A Robust Global Router for Modern Designs," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 29, no. 12, pp. 1931-1944, 2010.